PhysicalData Network: PRD v2

Oct 10, 2026 · @Shreyash Rajesh Raut

1. Overview

PhysicalData Network lets a robotics team post image and video labelling jobs, have qualified contributors label them for stablecoin rewards, and pay only for work that passes review. Monad holds the budget and the payout receipts. Raw footage and labels stay in private storage.

Pitch line: Labels you can trust, because the checks are layered, the audits are random, and every payout is tied to a receipt.

What this version adds to PRD v1:

- Two data types, images and video, on one shared backend and one shared contract.
- Kimi (Moonshot AI) reviews image labels first. Humans review video and the hard cases.
- Privy is used as the signing and money-control layer, not only for login.
- Chainlink VRF picks random audit samples and hidden gold tasks, so nobody can predict or fix them.
- A FAQ at the end answers every question the team asked while planning.

2. The problem

Robots learn from images and video, but raw footage is not training data. Someone has to mark what is in each image, and when each action starts and ends in a video, and then someone has to check that work.

Who	What hurts today
Robotics team (buyer)	Labels are noisy and inconsistent. Reviewing bad labels costs time. They cannot see who did the work, which rules applied, or who checked it. Data rights are often unclear.
Contributor	Instructions are vague. Work is rejected with no reason. Payment is late, or fees eat small payouts. There is no appeal.

Who	What hurts today
Reviewer or operator	Review is manual and slow. Fake and duplicate accounts are hard to spot. There is no clean audit trail.

Why the usual options fall short

- Managed labelling vendors give quality, but little visibility and a rigid, expensive process.
- Open crowd marketplaces bring supply, but also fraud, noisy labels and privacy risk. The buyer ends up doing the QA.
- Onchain payments alone make payout fast and transparent, but paying someone does not prove the label is correct.

Our position: we sell accepted, reviewed, export-ready labels with a receipt for every payout. We do not sell decentralisation. The chain proves workflow and payment state. Quality checks and review prove correctness.

3. Scope

We are building the full loop for two data types: create and fund a job, label, review, pay, export. The contract is the same for both. Only the labelling screen, schema and review route change.

Area	In scope	Out of scope
Data types	Images (boxes, simple answers). Video (timestamped action and failure segments).	LiDAR, point clouds, depth fusion, multi-sensor sync
Contracts	Escrow, accepted-task receipts, claim, pause, refund rules	Token launch, staking, DAO, cross-chain
Auth	Email or wallet login, four roles, server-side role checks	Full KYC, proof of humanity
Review	Automatic checks, gold tasks, Kimi first pass for images, human review for video and hard cases, random audits	Fully autonomous AI acceptance
Output	JSONL and CSV export of accepted labels with a provenance manifest	Dataset marketplace

Area	In scope	Out of scope
Users	Invite-only contributor cohort, one buyer	Open public uploads

Contributors never write code. They watch, draw and click.

4. Users and roles

Four roles, each checked on the server for every request.

Role	What they do	Cannot do
Buyer	Creates and funds jobs, sets the rubric, downloads the export, confirms delivery	Review labels or change accepted results
Contributor	Passes qualification, claims tasks, labels, sees status and rewards, claims payouts, appeals	Review any work, including their own
Reviewer	Accepts, requests revision or rejects with a reason code, handles appeals	Review their own submissions or move funds
Admin	Pauses jobs, manages roles, handles disputes, operates the signer policy	Silently take funds, or edit accepted labels

Wallet ownership proves control of a key, not a unique human. We do not claim Sybil resistance.

5. Chain choice: Monad

We build on Monad because the hackathon requires it, and because it fits a product that needs many small, cheap, fast settlements. Monad is a parallel EVM chain, so Solidity, Foundry, Wagmi and Viem work as usual.

Why Monad fits this product

- **EVM compatible:** the team uses tools it already knows, which matters with about 2.5 days left.
- **Fast and cheap transactions:** we write a receipt per accepted task and pay small rewards. High fees would eat a small payout. We still batch payouts where we can. Check live fee and block-time numbers on the Monad docs before quoting them in the pitch.

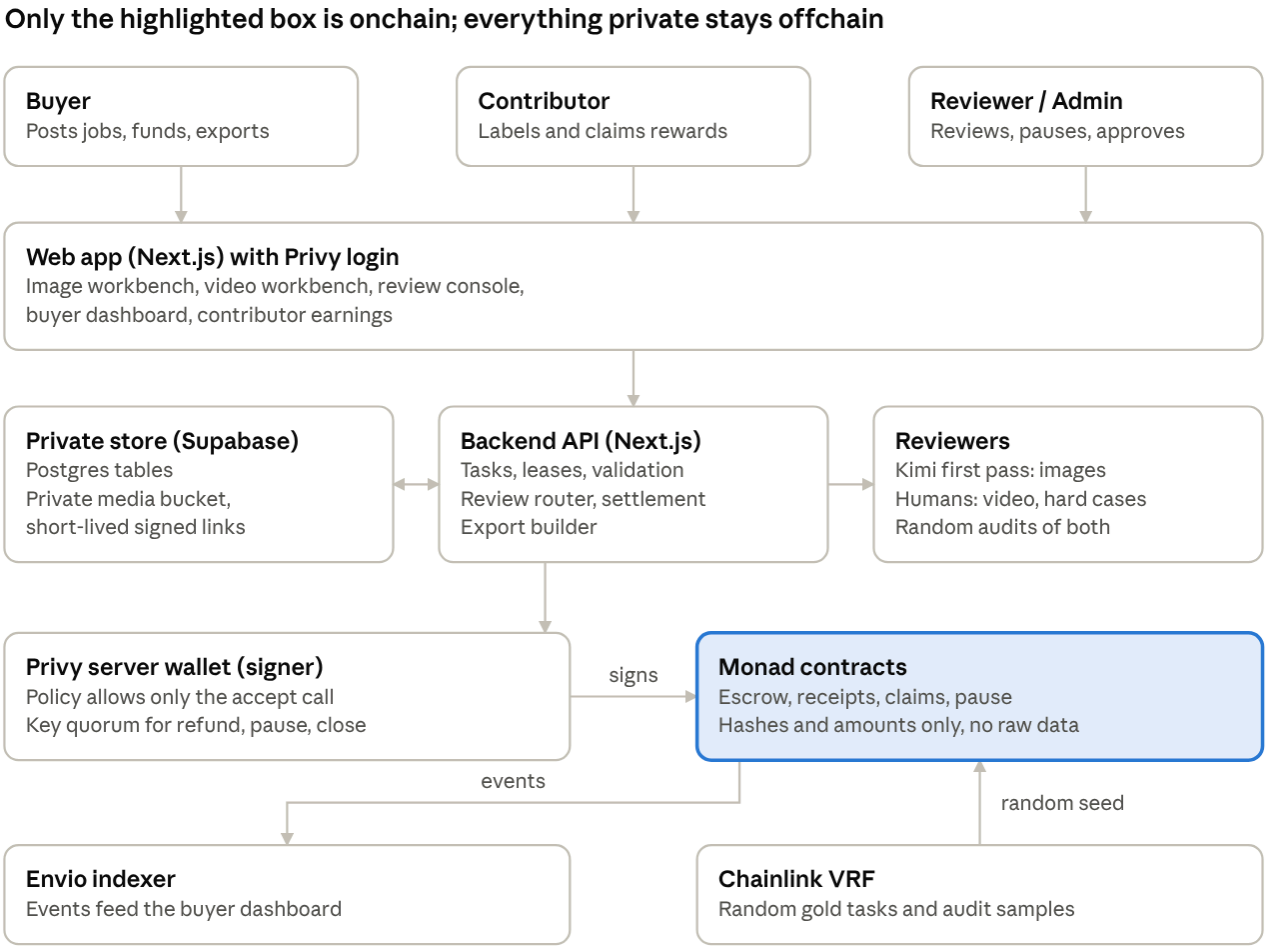
- **Sponsor ecosystem:** Privy, Chainlink, Envio, Agora and others are listed as Metropolis sponsors, so the tools we want sit next to the chain.
- **Testnet for the demo:** rewards are test tokens with no real value, and we say so on screen.

What goes onchain and what does not

Onchain (Monad)	Offchain (private)
Job ID, buyer address, budget, policy hash, status	Raw images and video
Escrowed funds and amounts paid	Full labels and reviewer notes
Hash of each accepted task, contributor, policy version	Task instructions, schemas, QA output
Payout claimable and claimed events, pause state	Contributor profiles, appeals, consent and rights records

No raw media, personal data or full label ever enters a transaction or event. A hash proves a record was not changed. It does not prove the label is right.

6. Architecture

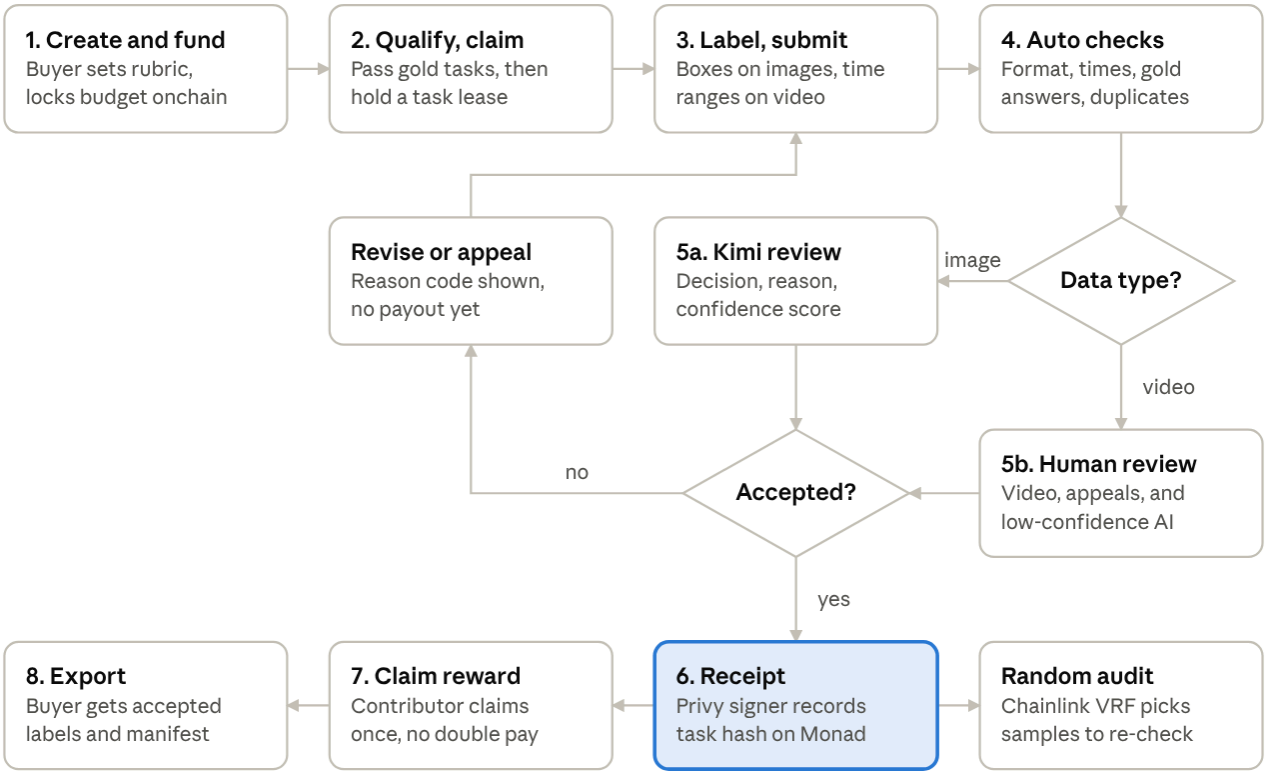


system architecture · 11 components

Money and receipts live only on Monad. The Privy signer is the single path that writes acceptances, and Chainlink supplies the random seed for audits. Media, labels and profiles never leave the private store.

7. End-to-end flow

Only accepted work reaches payout; rejected work goes back to the contributor



end-to-end flow · 8 steps, 2 decisions

Rejected or revise-needed work loops back to the contributor with a reason and is never paid. Low-confidence Kimi decisions go to the human queue, and Chainlink picks random accepted work for a human re-check.

Task states

State	Meaning	Payout
Draft	Contributor is still editing	None
Submitted	Passed client checks, waiting in the queue	None
Needs revision	Fixable problem with a rubric reason	None until accepted
Accepted	Passed automatic checks and review	Eligible to be paid
Rejected	Failed the stated criteria	Not paid unless an appeal reverses it

State	Meaning	Payout
Appealed	Contributor disputes the result	Pending for the disputed part
Paid	Settlement confirmed by contract receipt	Done

8. Contributor workflow

A contributor does careful data entry in a browser. No coding, no screenshots. They label a time range in a video, or draw on an image, and the tool saves the numbers.

Common steps

1. Sign in (email or wallet) and read the job rules, reward, deadline and how work gets accepted.
2. Pass a short qualification on clips or images whose right answers are already known.
3. Open the task queue, check reward and effort, and claim a task. A lease holds it for one contributor so nobody else submits the same task.
4. Label the item (see below). The tool autosaves a draft.
5. Submit. The tool checks the format and tells them to fix problems right away.
6. See the result: accepted, needs revision or rejected, with a plain reason. Accepted work becomes claimable.
7. Claim the reward and view the receipt.

Video example: someone writing on paper (12 seconds)

The contributor does not capture a single frame. They mark the time range of each action on a timeline.

Start	End	Label	Notes
0.0 s	1.2 s	reach	Hand moves toward the pen
1.2 s	3.4 s	grasp	Pen is held, outcome success
3.4 s	4.0 s	lift	Pen leaves the table
4.0 s	11.0 s	write	Needs adding to the label list
11.0 s	12.0 s	place	Pen set down

How they do it: play the clip once fully, step through frame by frame to find the exact start and end, drag on the timeline or type the times, pick the label button, then fill object, outcome (success, partial, failure, unknown) and confidence (1 to 5). If the pen slips at 2.8 s, they mark a failure segment with reason slip, then the recovery. If unsure, they pick unknown or other instead of guessing.

The PRD v1 label list was reach, grasp, lift, place, inspect and recover. Add write if the demo uses writing.

Image example: robot camera view of a table

The contributor draws a box around each object (cup, pen, red block), picks the class from a list, and optionally answers a short question such as whether the grasp succeeded. Boxes are saved as pixel coordinates.

Rules the tool enforces

- Video segments follow each other with no overlap, and times stay inside the clip.
- Boxes stay inside the image and have a class.
- Precise start and end times matter most in video, because that is what the buyer pays for and the reviewer checks.

9. Quality and judging

The buyer defines what correct means. The system and reviewers apply it, and the buyer only sees accepted work. Contributors never judge their own work, and a reviewer can never review their own submission.

Who judges what

Judge	Does what	Used for
Buyer	Writes the rubric: label meanings, precision needed, what counts as correct. No testable rubric, no launch.	Ground truth
Code	Schema checks, time and box validation, gold-task scoring, duplicate detection	Every submission
Kimi (Moonshot AI)	Suggests accept, revise or reject with a reason code and a confidence for image labels	Images, first pass
Human reviewer	Final call on video, low-confidence AI decisions, appeals and audits	Video and hard cases

Judge	Does what	Used for
Chainlink VRF	Picks random gold tasks, reviewer assignment and audit samples, recorded on Monad	Anti-gaming

The layers, in order

1. **Automatic checks:** end before start, times outside the clip, overlaps, missing fields, boxes outside the image. Rejected instantly with a reason.
2. **Gold tasks:** hidden items with known answers, in qualification and mixed into live queues. Boxes are scored by overlap (IoU), segments by time difference and label match.
3. **Overlap sampling:** some items go to 2 or 3 contributors. Agreement raises confidence. Disagreement goes to a reviewer.
4. **First-pass review:** Kimi reviews image submissions against the rubric. Video goes straight to the human queue.
5. **Risk-based human review:** low-confidence, first-time contributors, unusually fast submissions and gold-task failures come first.
6. **Random audits:** a random sample of accepted work, including AI-decided work, is re-checked by a human. We track how often the human disagrees with Kimi. That number is our evidence that AI review works.

Routing rule

- High-confidence Kimi decision: applied, with the reason shown to the contributor.
- Low confidence, or the contributor appeals: goes to a human.
- Every decision is marked AI-suggested or human-reviewed in the UI and in the export manifest.

Why Kimi is not the only judge: video timing needs frame-level precision, where AI vision is weak. Buyers will not accept an AI approval as proof by itself. A wrong AI approval would mean paying for bad work. Check Kimi's current docs for image input support and which model to call.

Contributor reputation: based on gold accuracy, agreement with reviewers, revision rate and appeal outcomes. It is shown only with enough data, and never as a tradable token.

States: Draft, Submitted, Needs revision, Accepted, Rejected, Appealed, Paid. Rejected work is never paid unless an appeal reverses it.

10. Privy: beyond login

The Privy bounty says login-only integrations do not qualify, so Privy is our signing and money-control layer. Privy's docs describe three layers: authentication, wallets, and controls (owners, signers, policies). We use all three.

Use 1 (core): the acceptance signer is a Privy server wallet

The contract only marks work payable when the authorised signer calls it. In PRD v1 that key sat in an environment secret, which the PRD itself calls weak. Now the key lives in a Privy server wallet controlled by an authorization key. The backend asks Privy to sign each `markAccepted` call. Privy's policy engine runs before anything is signed, and policies deny everything by default. We allow one action: calls to our JobEscrow contract and the accept function only.

If removed: the signing key sits in a `.env` file, and whoever steals it can mark fake work as accepted.

Use 2: key quorum for risky actions

Routine accepted-work calls run automatically under the policy. Sensitive actions (refund, close job, pause, change policy) need approval from more than one key in a quorum, for example the buyer and an admin. Privy documents this pattern as automatic execution for routine transactions with quorum sign-off for sensitive ones. No single person can drain the escrow.

Use 3 (bonus): gasless claims

Contributors claim rewards without holding gas tokens, using Privy gas sponsorship. If it is not supported on Monad, the Privy server wallet pushes payouts to contributors instead.

Login still uses Privy (email or wallet, embedded wallet for contributors), but that is not what we claim for the bounty.

Pitch line: no single key can approve fake work or move the escrow, and every payout goes through a signer limited by policy.

Verify before building: Privy supports Monad testnet for server wallets, policies and gas sponsorship; policies can restrict by contract address and function on our chain; and the dashboard bounty text for judging criteria. The Mera bounty from the Monad Foundation needs Mera as the whole account layer, which conflicts with Privy. Pick one.

11. Other sponsors

Sponsor names come from public listings of the Metropolis hackathon. The dashboard blocked automated access, so read each bounty's exact terms there before committing. Test for each integration: if we delete it, does something real in the product break?

Sponsor	How it plugs in	If removed, what breaks	Effort	Verdict
Privy	Server wallet signer, policies, key quorum, gas sponsorship (section 10)	Signing key falls back to an env secret	Medium	Core
Chainlink (VRF)	Random gold tasks, random reviewer assignment, random audit sample, result stored on Monad so anyone can check the sample was not hand-picked	We would choose audits ourselves, which is a trust gap	Medium	Strong fit, headline
Kimi (Moonshot AI)	First-pass image review with a reason and confidence (section 9)	Humans review every image	Medium	Good fit, check bounty terms
Envio	Indexes JobCreated, TaskAccepted and PayoutClaimed events for the buyer dashboard and reconciliation	We hand-build the reconciliation worker	Low	Easy add
Agora (stablecoin)	Pay rewards in their stablecoin instead of a mock token	Rewards fall back to a test token	Low if the token exists on Monad testnet	Nice to have
Nansen	Wallet analytics as one signal for suspicious accounts	Nothing in the MVP breaks	High	Skip for now

To verify: Chainlink VRF availability on Monad testnet (fallback: a clearly labelled commit-reveal seed, and skip that bounty); Kimi image input and its bounty terms; Agora token on Monad testnet.

12. Smart contracts

Keep the contract surface small and modality-agnostic. The contract sees a task ID hash, an annotation hash, a contributor and a reward. It never knows whether the work was an image or a video.

Component	Job
JobEscrow	Create and fund jobs, hold the budget, track reserved and paid amounts, close or refund under published rules
Acceptance and payout (same contract for the MVP)	Record accepted task hashes with policy version, make rewards claimable, let contributors claim once
Access control and pause	Roles for the Privy signer and admin, emergency pause, quorum-gated sensitive actions
Audit randomness	Requests Chainlink VRF, stores the result, so audit samples can be re-derived by anyone

Invariants (must always hold, covered by tests)

- Total paid never exceeds the funded budget.
- A task ID hash can be accepted and paid only once.
- Only the authorised Privy signer can mark work payable.
- A paused or closed job takes no new claims or acceptances.
- Refunds cannot touch money already owed to contributors.
- Admin actions emit events and are access-controlled.
- Raw data and personal information never appear in calldata or events.

Events: JobCreated, JobFunded, TaskAccepted (job, task hash, contributor, annotation hash, policy version, reviewer type: AI or human), PayoutClaimable, PayoutClaimed, JobPaused, JobClosed, PolicyUpdated, AuditSeedReceived.

MVP simplification: per-task claims are fine for the demo. Merkle-root batched payouts come later.

Trust assumptions to say out loud: acceptance is signed by our QA service through the Privy signer. That is a deliberate central point, now limited by policy and quorum. A hash proves integrity, not correctness. Testnet tokens have no real value.

13. Data model and API

One `tasks` table with a `modality` field (image or video) and one `submissions` table with the label payload as JSON keep the backend shared.

Entity	Key fields
User	id, walletAddress, role (buyer, contributor, reviewer, admin), status
ContributorProfile	userId, qualification result, quality stats, consent version
Project	id, buyerId, data rights, retention policy
Asset	id, projectId, objectKey, contentHash, modality, durationMs (video), width and height (image), consentRef
SchemaVersion	id, modality, JSON schema, label list, versionHash (immutable once published)
Job	id, projectId, budget, reward per accepted task, deadline, policyHash, status, chain job ID
Task	id, jobId, assetId, modality, leaseOwner, leaseExpiry, status, isGold
Submission	id, taskId, contributorId, payload JSON, payloadHash, submittedAt (versioned, never overwritten)
Review	id, submissionId, reviewerType (code, ai, human), reviewerId, decision, reasonCode, confidence, rubricVersion
Appeal	id, submissionId, reason, status, adjudicatorId
Receipt	id, submissionId, annotationHash, policyHash, reviewRef, txHash
AuditSample	id, jobId, vrfRequestId, seed, selected task IDs

API (illustrative)

Endpoint	Purpose	Who
POST /api/jobs, POST /api/jobs/{id}/publish	Create draft, publish after funding	Buyer
GET /api/tasks/available	List eligible tasks	Contributor
POST /api/tasks/{id}/claim	Claim a lease (idempotent)	Contributor
PUT /api/tasks/{id}/draft, POST /api/tasks/{id}/submit	Autosave, final submit with validation	Task owner
POST /api/reviews/{submissionId}/decision	Accept, revise or reject with reason code	Reviewer (not the author)
POST /api/submissions/{id}/appeal	Appeal a rejection	Contributor
POST /api/settlement/accept	Ask the Privy signer to record an accepted task onchain	Settlement service
GET /api/jobs/{id}/export	Accepted-only export and manifest	Buyer
GET /api/receipts/{id}	Minimal public receipt	Public

All input is validated on the server. Every route checks the Privy login token, then the role. Submissions and settlement use idempotency keys.

14. Annotation schemas and exports

Image schema `image-boxes-v1`

```
{
  "task_id": "task_01H...",
  "asset_id": "asset_private_017",
  "schema_version": "image-boxes-v1",
  "boxes": [
    { "x": 120, "y": 80, "w": 90, "h": 110, "class": "cup", "confidence": 4
},
    { "x": 300, "y": 210, "w": 40, "h": 40, "class": "red_block",
"confidence": 5 }
  ],
  "answers": { "grasp_success": "yes" }
}
```

Rules: boxes inside the image, positive size, class from the job's list. Scored against gold boxes by IoU (overlap).

Video schema `action-segments-v1`

```
{
  "task_id": "task_01H...",
  "asset_id": "asset_private_042",
  "schema_version": "action-segments-v1",
  "segments": [
    { "start_ms": 1200, "end_ms": 3400, "action_label": "grasp", "object":
"pen", "outcome": "success", "confidence": 4 },
    { "start_ms": 4000, "end_ms": 11000, "action_label": "write", "object":
"pen", "outcome": "success", "confidence": 4 }
  ]
}
```

Rules: start_ms is 0 or more, end_ms is greater than start_ms and inside the clip, segments in order with no overlap, failure_reason required when outcome is failure. Labels: reach, grasp, lift, place, inspect, recover, write, other.

Exports

- Images: COCO-style JSON. Video: JSONL for nested segments and CSV for flat rows.
- A manifest with dataset ID, schema version, asset hashes, rights reference, review policy version, reviewer type per record (AI or human), export time and record counts.
- Accepted records only. Never include worker email, legal name or other identity data.

15. UI and UX

Laptop first. It is a work tool, so keep it calm: one accent colour, a big work area, plain-language statuses. Mobile labelling is not in the MVP.

Screen	Must show	Main action
Landing	What buyers get, how contributors are paid, privacy stance	Create job or join cohort
Buyer job wizard	Data type, schema, rubric, budget, reward, deadline, data rights	Publish and fund
Buyer dashboard	Tasks done, review queue, acceptance rate, spend, budget left, AI vs human review share	Export or close job

Screen	Must show	Main action
Contributor onboarding	Sign in, rules, privacy note, sample task	Start qualification
Task queue	Reward, effort, deadline, data type	Claim task
Image workbench	Image, box tool, class palette, rubric, examples	Submit
Video workbench	Player with frame step, timeline, segment form, rubric	Submit
Review console	Item, rubric, submission vs reference, AI suggestion with reason and confidence, reason codes	Accept, revise or reject
Contributor earnings	Accepted, pending, rejected, appealed work, receipts	Claim payout
Receipt view	Task ID, policy hash, annotation hash, reviewer type, tx link	Verify
Admin	Pause, role management, signer policy status, quorum approvals	Pause or approve

Video workbench layout: top bar with task number, reward and lease timer. Video on the left with play, speed and frame-step controls, and the timeline underneath with coloured segments. Label buttons and fields (object, outcome, failure reason, confidence) on the right, with Save draft and Submit.

Review console layout: submitted labels shown next to reference labels, mismatches highlighted, rubric checklist on the right, required reason code, and three buttons: Accept, Request revision, Reject.

UX rules

- Show reward and rubric before the contributor claims a task.
- Use plain words for statuses and rejection reasons.
- Status colours: grey pending, amber needs revision, green accepted, red rejected, purple paid.
- Submitted, accepted and paid are separate states in the UI.
- Do not show a reputation score without enough data.
- Mark every label and decision as human-made, AI-suggested, reviewed or adjudicated.
- Every action has loading, error and retry states.

16. Tech stack

The stack is trimmed for a 2 to 3 day build: fewer services to set up and fewer places to break.

Layer	Choice	Why
Frontend	Next.js, React, TypeScript, Tailwind	One codebase for every screen
Image labelling	HTML canvas box tool (or a small open-source library)	Fast to build and easy to validate
Video labelling	HTML5 video, custom timeline, frame stepping	Needed for timestamped segments
Auth and wallets	Privy (login, embedded wallets, server wallet, policies, key quorum, gas sponsorship)	Section 10
Wallet and chain client	Wagmi and Viem	Reads, writes, receipts
Contracts	Solidity and Foundry	Easy tests for budget, double-pay and access
Backend	Next.js API routes (TypeScript)	No separate server to deploy
Database and storage	Supabase (Postgres and a private storage bucket)	Tables plus short-lived signed media links
AI review	Kimi (Moonshot AI) API for image first pass	Section 9
Randomness	Chainlink VRF	Random audits and gold tasks
Indexer	Envio HyperIndex (or a small Viem event listener)	Dashboard and reconciliation
Hashing	keccak256 over canonical JSON	Same hash on server and contract
Network and token	Monad testnet, test ERC-20 (or sponsor stablecoin)	Real transactions, no real money

17. Security, privacy and trust assumptions

The riskiest parts are money movement and private data. These rules apply from day one.

- **Private media:** buckets are never public. Playback uses signed links that expire in minutes, and every link request checks role and job.
- **Onchain minimum:** hashes, amounts and state only. No raw data, personal data or full labels in calldata or events.
- **Signing key:** held by Privy, limited by policy to our contract's accept function, with quorum approval for refund, close, pause and policy change.
- **Roles:** checked on the server for every route. Contributors cannot see reviewer-only data. Reviewers cannot review their own work.
- **Fraud controls:** rate limits, task leases, idempotency keys, duplicate-payload detection and unusually fast submissions flagged for review. A flag triggers a human look, not automatic punishment.
- **AI review:** Kimi only suggests. Decisions are marked as AI, high-confidence ones are audited at random by humans, and any contributor can appeal to a human.
- **Data rights:** the buyer attests it may share the data. Use synthetic or permission-cleared media for the demo.
- **Money:** testnet only. Real stablecoin payouts need legal, tax and token review first.

Honest limits to state in the pitch

- Acceptance is signed by our QA service. It is policy-limited, but still central.
- A hash proves a record was not changed. It does not prove a label is correct.
- Wallet ownership is not proof of a unique person, so we make no Sybil-proof claim.
- AI review reduces human work but is not a perfect judge.
- The contract code is not audited.

18. Build plan

The submission deadline is listed as 12 or 13 October 2026. The rules page says 13 October, 11:59 PM ET. Today is 10 October, so there are about 2.5 days. Confirm the exact time on the portal.

Order of work

Block	Build	Done when
1	Contracts and tests, Privy login and roles, database schema	Budget, double-pay and access tests pass. Login works for all roles
2	Image flow end to end, including Kimi review	One image goes from created to paid

Block	Build	Done when
3	Video flow, reusing everything from block 2	One clip goes from created to paid
4	Privy server wallet signer and policy, settlement, export, buyer dashboard	Accepted task is signed by the policy-limited signer. Export downloads
5	Chainlink VRF for audits and gold tasks, Envio indexer	Random result visible on Monad and used by the app
Last 6 hours	Demo script, error states, README, demo video, clean-browser test	Full flow works from a fresh browser

Team lanes (agree database schema and API shapes in the first hour)

- Lane 1: contracts, settlement, Chainlink.
- Lane 2: Privy auth, signer, policies, backend and database.
- Lane 3: image workbench and Kimi review.
- Lane 4: video workbench and review console.

Checkpoint: by the end of day 1, the image flow should run from job creation to payout. If not, drop video AI, appeals and Envio, and use plain wallet connect for login.

Cut list if time runs out: appeals, reputation score, consensus labelling, batched payouts, gas sponsorship, Agora token, Nansen, extra schemas.

19. Demo script and fallback

One story in under four minutes. Use synthetic or permission-cleared media only.

#	What you do	What judges see
1	Buyer creates a job with rubric and reward	A real task, not a static page
2	Fund the job with the testnet token	Budget locked on Monad, tx link
3	Contributor signs in with Privy and passes a gold task	Quality is separate from owning a wallet
4	Contributor draws boxes on an image	Image workflow
5	Kimi reviews it and returns a decision, reason and confidence	AI first pass, marked as AI

#	What you do	What judges see
6	Contributor labels a video clip (reach, grasp, failure)	Robotics-specific timeline workflow
7	Submit a bad label (end before start)	Rejected with a reason, no payout
8	Reviewer accepts a good video label	Receipt with hash, policy version, reviewer type
9	Privy server wallet signs the acceptance, contributor claims the reward	Policy-limited signer, contract event, no double claim
10	Show the Chainlink random audit result on Monad	Checks are random and verifiable
11	Buyer downloads the export	Real files plus manifest

Fallback: preload a sample job for the walkthrough but still show at least one live transaction. If a transaction fails, show the error. Do not fake success. Keep a recording of a good run, labelled as a recording. One complete flow beats many half-working screens.

20. Risks and open questions

Risk or question	Severity	Plan
Scope: two data types in about 2.5 days	High	Shared backend and contract. Image flow first. Day 1 checkpoint
Privy does not support Monad testnet for server wallets, policies or gas sponsorship	High	Verify first. Fall back to a plain signer and drop that bounty
Chainlink VRF not on Monad testnet	Medium	Labelled commit-reveal fallback, skip that bounty
Kimi image input or model limits	Medium	Check docs. Fall back to human review for all images
Kimi approves bad labels	High	Confidence threshold, random human audits, appeal to a human
Fake or duplicate contributors	High	Qualification, leases, anomaly flags, no Sybil-proof claim

Risk or question	Severity	Plan
Payout bug or contract exploit	Critical	Small contract, tests, capped demo funds, pause, quorum for sensitive actions
Privacy leak	Critical	Private bucket, expiring links, role checks
Mera and Privy bounties conflict	Low	Pick one
No buyer pull after the hackathon	Critical	Customer discovery with real clips before building more

Open questions: exact deadline time on the portal; exact bounty terms for Privy, Chainlink, Kimi, Envio and Agora; whether Agora's stablecoin exists on Monad testnet.

21. FAQ: the questions we asked

What problem are we solving, in simple words? Robots need huge amounts of labelled images and video. Big labelling companies take a large cut, workers get paid slowly or little, and buyers cannot see who checked what. We make the checks layered and visible, and we pay for accepted work through a contract with a receipt per payout.

What is the pain point, and how do we solve it? The core pain is fake and junk labels. We use automatic checks, hidden gold tasks, Kimi first-pass review for images, human review for video and hard cases, and random audits picked with Chainlink VRF. Payment happens only after acceptance.

What makes us different from other teams? One sponsor integration that fixes a real trust problem: a Privy server wallet signer limited by policy and quorum, plus random audits from Chainlink VRF. Most teams will only add a login.

Can Claude do this work if we buy a Claude plan? Claude Code can help build the software: contracts, tests, frontend, backend and database. As of the support docs we read, Pro and Max plans cover Claude Code, and usage limits are shared between Claude chat and Claude Code. After the limit you can enable extra usage, billed at API rates, or wait for the reset. If an ANTHROPIC_API_KEY variable is set, Claude Code uses it and bills the API instead of the plan. You still do the bounty checks, testnet funds, key handling, contract review, demo recording and the final testing.

Can Claude do the actual labelling, or train the robots? Claude can describe what it sees in images, suggest labels and flag likely mistakes. It is weak at frame-exact timestamps, which robotics buyers pay for, and it does not train robot models (training runs on your

own compute). So AI suggests and humans confirm. A buyer will not accept AI-only labels as final.

Does a contributor have to code? No. They watch a clip or look at an image, draw boxes or mark time ranges on a timeline, pick labels from buttons and dropdowns, and submit. See section 8.

If the clip shows someone writing on paper, do contributors take screenshots? No. They label the time range of each action. For example, 1.2 s to 3.4 s is grasp, and the tool stores the timestamps. If the action is not in the label list, add it (write).

How does the company know the work is genuine, and who judges when many people work on one project? The buyer writes the rubric. Code checks every submission, gold tasks catch careless work, overlap sampling finds disagreement, reviewers decide the hard cases, and random audits check the reviewers and the AI. The buyer receives accepted labels only, with a record of who contributed and which review passed. See section 9.

Can Kimi (Moonshot AI) do the judging instead of our team? As a first-pass reviewer, yes. It suggests a decision with a reason and a confidence. It should not be the only judge: video timing needs precision, buyers want more than an AI approval, and a wrong approval means paying for bad work. We keep human review for low confidence, appeals and random audits.

Can we use Kimi if the work is about images instead of videos? Yes, and images suit AI review better because there is no frame-exact timing. Box overlap is scored by code. Kimi judges meaning, such as whether a box really contains a cup. Check Kimi's docs for image input before building.

Can we use Kimi for images and our team for videos? Yes, and that is the plan. A routing rule sends images to the Kimi queue and video to the human queue. Because the backend and contract are shared, the second data type is mostly a new workbench screen.

Can we build everything end to end: contracts, images, video and auth? Yes, if the backend and contract stay shared and the work is done in order: contracts and auth first, image flow second, video third, then settlement, then Chainlink and Envio. Set a day 1 checkpoint: the image flow must reach payout. See section 18.

The Privy bounty says login-only integrations do not qualify. What is a better use? Make Privy the signing and money-control layer. The acceptance signer becomes a Privy server wallet with a policy that allows only our contract's accept function. Sensitive actions need a key quorum. Gasless claims are a bonus. See section 10.

Sources consulted

- Using Claude Code with your Pro or Max plan

- Privy treasury overview and Privy docs reference
- Monad Metropolis and Metropolis listing on Rise In, read as search results on 10 October 2026